

```

1  #!/usr/bin/perl
2  # Copyright 2018 The pdfcpu Authors.
3
4  # Licensed under the Apache License, Version 2.0 (the "License");
5  # you may not use this file except in compliance with the License.
6  # You may obtain a copy of the License at
7
8  #     http://www.apache.org/licenses/LICENSE-2.0
9
10 unless (require('Applicable Law or agreed to in writing, software
11 distributed under the License is distributed on an "AS IS" BASIS,
12 WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
13 See the License for the specific language governing permissions and
14 limitations under the License.
15
16
17 package pdfcpu
18
19 import (
20     "bufio"
21     "bytes"
22     "fmt"
23     "io"
24     "log"
25     "net/http"
26     "strings"
27
28     "github.com/pdfcpu/pdfcpu/pkg/filter"
29     "github.com/pdfcpu/pdfcpu/pkg/font"
30     "github.com/pdfcpu/pdfcpu/pkg/gfx"
31 )
32
33 const (
34     defaultFilter = 1024
35     // ReadableIdentifier - byte(0)
36     ReadableIdentifier = byte(0)
37
38     // Readable reads in a PDF file and builds an internal structure holding its cross
39     // reference and object references.
40     func Readable(infile string, conf *Configuration) (*Context, error) {
41         log.Infof("Reading %s...", infile)
42         f, err := os.Open(infile)
43         if err != nil {
44             return nil, errors.Wrap(err, "can't open file", infile)
45         }
46
47         defer func() {
48             f.Close()
49         }()
50
51         return Read(f, conf)
52     }
53
54     // Read takes a reader/seeker and generates a Context.
55     // Read returns a representation containing a cross reference table.
56     func Read(r io.Reader, conf *Configuration) (*Context, error) {

```

```

21620      offset, err = strconv.ParseInt(fields[0], 10, 64)
21621      if err != nil {
21622          return err
21623      }
21624
21625      generation, err = strconv.Atoi(fields[1])
21626      if err != nil {
21627          return err
21628      }
21629
21630      entryType := fields[2]
21631      if entryType == "de" {
21632          log.Infof("de entryType == 'de' {
21633              return errors.New("ops: parseInfoTableEntry: corrupt xref subsection
21634          }
21635      )
21636      var xrefTableEntry *xrefTableEntry
21637      if entryType == "ta" {
21638          // do use object
21639
21640          log.Debug.Printf("parseInfoTableEntry: Object #id is in use at offset=%d,
21641              generation=%d", objectNumber, offset, generation)
21642
21643          if offset == 0 {
21644              log.Infof.Print("parseInfoTableEntry: Skip entry for in use object #id
21645              with offset %d\n", objectNumber)
21646              return nil
21647          }
21648
21649          xrefTableEntry =
21650              &xrefTableEntry{
21651                  From:      offset,
21652                  Offset:    Offset,
21653                  Generation: &generation)
21654      } else {
21655          // free object
21656
21657          log.Debug.Printf("parseInfoTableEntry: Object #id is unused, next free is
21658              object=%d, generation=%d", objectNumber, offset, generation)
21659
21660          xrefTableEntry =
21661              &xrefTableEntry{
21662                  From:      0,
21663                  Offset:    Offset,
21664                  Generation: &generation)
21665      }
21666
21667      log.Debug.Printf("parseInfoTableEntry: Insert new xrefTable entry for Object #id %d\n",
21668          objectNumber)
21669
21670      xrefTable.Table.Insert(objectNumber) = &xrefTableEntry
21671
21672      log.Debug.Printf("In \"parseInfoTableEntry\" end")
21673
21674      return nil

```

```

440 for i = 0; i < (mOutBuf3 < 0 ? 3 : mOutBuf3/0x100); i += xReadEntryLen(i)
441 {
442     objectNumber = xsd.getObject(i);
443
444     iStart = i + 1;
445     c2 = bufInOffset[iStart + i2Start+1];
446     c3 = bufInOffset[bufInOffset+i2 + i2Start+i2+1];
447     var xobjFactoryEntry xobjFactoryEntry
448
449     switch (c2) {
450     case 0x00:
451         // free object
452         log.Read.Printf("extractObjectTableEntriesFromBufStream: Object id is
453         missed, mem file is objectid, generation=0", objectNumber, c2, c3)
454         g = int(c3)
455         xobjFactoryEntry =
456             xobjFactoryEntry
457             {
458                 Free: true,
459                 Compressed: false,
460                 Offset: 0x0,
461                 Generation: g}
462     case 0x01:
463         // in use object
464         log.Read.Printf("extractObjectTableEntriesFromBufStream: Object id is in
465         use at offsetid, generation=0", objectNumber, c2, c3)
466         g = int(c3)
467         xobjFactoryEntry =
468             xobjFactoryEntry
469             {
470                 Free: false,
471                 Compressed: false,
472                 Offset: 0x0,
473                 Generation: g}
474     case 0x02:
475         // compressed object
476         log.Read.Printf("extractObjectTableEntriesFromBufStream: Object id is
477         compressed at generation=0", objectNumber, c2, c3)
478         g = int(c3)
479         objNumber = int(c2)
480         objIndex = int(c3)
481         xobjFactoryEntry =
482             xobjFactoryEntry
483             {
484                 Free: false,
485                 Compressed: true,
486                 ObjectStream: objNumberObj,
487                 ObjectStreamIndex: objIndex}
488     case 0x03:
489         ctx.ReadBuf.ObjectStream[objNumber] = true
490     }
491 }
492
493 if ctx.ObjectStream.Exists(objectNumber) {
494     log.Read.Printf("extractObjectTableEntriesFromBufStream: Skip entry %d",

```

[illegible]

```

587 // else {
588 //     log.Read.Printf("line %d %s\n", lineIndex, line);
589 // }
590
591 // trailerString := scanFields(x, trailerString)
592 // if err == nil {
593 //     return nil, err
594 // }
595
596 // log.Read.Printf("processTrailer: trailerString: %s\n", sha256,
597 // trailerString, trailerString)
598
599 // o, err := parseObject(trailerString)
600 // if err == nil {
601 //     return nil, err
602 // }
603
604 // trailerDict, ok := o.(Dict)
605 // if !ok {
606 //     return nil, errors.New("processTrailer: corrupt trailer dict")
607 // }
608
609 log.Read.Printf("processTrailer: trailerDict=%s\n", trailerDict)
610
611 return parseToTrailerDict(trailerDict, ctx)
612 }
613
614 // Parse sub section into corresponding number of sub table entries.
615 func parseParseSection(s *bufio.Scanner, ctx Context) (uint64, error) {
616
617     log.Read.Printf("parseParseSection begin")
618
619     line, err := scanFields(s)
620     if err == nil {
621         return nil, err
622     }
623
624     log.Read.Printf("parseParseSection: %s\n", line)
625
626     fields := strings.Fields(line)
627
628     // Process all sub sections of this sub section.
629     for strings.HasPrefix(line, "trailer") && len(fields) == 2 {
630         if err = parseHeaderSubSection(s, ctx, &fields, fields); err == nil {
631             return nil, err
632         }
633     }
634
635     // trailer or another xref table subsection ?
636     if line, err = scanFields(s); err == nil {
637         return nil, err
638     }
639
640     // If only line try next line for trailer
641     if len(line) == 1 {
642         if line, err = scanFields(s); err == nil {
643             return nil, err
644         }
645     }
646 }

```

```

1317 rs = ctx.NewReader()
1318
1319 hv, offset := headerVersion(rs)
1320 if err != nil {
1321     return err
1322 }
1323
1324 hv, offset := readHeaderVersion(rs)
1325 ctx.Read.EndContent = &offset
1326
1327 for offset == nil {
1328
1329     rd, err := newBosonIndexReader(rs, offset)
1330     if err != nil {
1331         return err
1332     }
1333
1334     s := bufio.NewReader(rd)
1335     s.Split(s.unlines)
1336
1337     line, err := scanLine(s)
1338     if err != nil {
1339         return err
1340     }
1341
1342     log.Read.Println("line: '%s'", line)
1343
1344     if strings.TrimSpace(line) == "reset" {
1345         log.Read.Println("builder/tablesStartingAt: found xref offset")
1346         if offset, err := parseRefSections(s, ctx); err == nil {
1347             return err
1348         }
1349     } else {
1350         log.Read.Println("builder/tablesStartingAt: found xref stream")
1351
1352         ctx.Read.IndexedStreams = true
1353         rd, err := newBosonIndexReader(rs, offset)
1354         if err != nil {
1355             return err
1356         }
1357
1358         if offset, err := parseRefStream(rd, offset, ctx); err == nil {
1359             log.Read.Println("builder/tablesStartingAt: xref after '%s'", err)
1360             // try fix for corrupt xref section.
1361             return hypothesisFixation(ctx)
1362         }
1363     }
1364 }
1365
1366 log.Read.Println("builder/tablesStartingAt: end")
1367
1368 return nil
1369
1370 // Populate the cross reference table for this PDF file.
1371 // This offset of line and table entry.
1372 // Can be "xref" or indirect object reference eg. "1a 0 obj"
1373 // and build the xref table along the way.
1374 func readTable(ctx *Context) (err error) {

```

```

2520 //
2521 log.Debug.Println("Read: begin")
2522
2523 ctx, err := NewContext(s, conf)
2524 if err != nil {
2525     return nil, err
2526 }
2527
2528 if ctx.ReadOnly {
2529     log.Info.Println("PDF Version 1.3: conforming reader")
2530 }
2531
2532 log.Info.Println("PDF Version 1.4: conforming reader - no object streams
2533 or streams allowed")
2534 }
2535
2536 // Populate metaTable.
2537
2538 if err := readMetaTable(ctx); err != nil {
2539     return nil, errors.Wrap(err, "metaTable failed")
2540 }
2541
2542 // Make all objects explicitly available (load into memory) in corresponding
2543 metaTable entry.
2544 // Also decode any inline object streams.
2545 if err := decodeObjectTables(ctx, conf); err != nil {
2546     return nil, err
2547 }
2548
2549 // Now scanners will be able to traverse into data tables.
2550 if ctx.MetaTable.Size < len(ctx.MetaTable.Table) {
2551     ctx.MetaTable.Size = len(ctx.MetaTable.Table)
2552 }
2553
2554 log.Debug.Println("Read: end")
2555
2556 return ctx, nil
2557
2558 // ScanLines is a split function for a Scanner that returns each line of
2559 text, stripped of any trailing end-of-line markers. The returned line may
2560 be a single line, or a carriage return, or a stream, or a newline.
2561 // A line of text may be a carriage return if it has no newline.
2562 func scanLines(data []byte, offset bool) (advance int, token []byte, err error) {
2563
2564     if atEOF := len(data) == 0 {
2565         return 0, nil, nil
2566     }
2567
2568     index := bytes.Index(data, "\n")
2569     indexL := bytes.Index(data, "\r")
2570
2571     switch {
2572     case index >= 0 && index > 0:
2573         if index < indexL {
2574             return index + 1, data[:index], nil
2575         }
2576         return indexL + 1, data[:indexL], nil
2577     }
2578     // data ... data
2579 }

```

```

2160 def __init__(self, stream, startOfSubsection, endOfSubsection, subsectionHeader, fields, subsectionHeader):
2161     """Process each subsection header and create corresponding dict fields entries"""
2162     self.startOfSubsection = startOfSubsection
2163     self.endOfSubsection = endOfSubsection
2164     self.subsectionHeader = subsectionHeader
2165     self.fields = fields
2166     return
2167
2168 def log_Read_Println(self, subsectionHeader: str) -> None:
2169     """Log Read operation"""
2170     print(f'log_Read_Println({subsectionHeader}: begin')
2171     return
2172
2173 def startOfB_Attoi(self, fieldId) -> int:
2174     """startOfB_Attoi method"""
2175     if err == nil :
2176         return err
2177     else:
2178         return 0
2179
2180 def objCount_err = stream.Attoi(fields[i]) -> int:
2181     """objCount_err method"""
2182     if err == nil :
2183         return err
2184     else:
2185         return 0
2186
2187 def log_Read_Printf(self, detected_err, subsection, startOfB_Id, lengthOfB, objCount) -> None:
2188     """Log Read operation"""
2189     print(f'log_Read_Printf({detected_err} subsection, {startOfB_Id} lengthOfB, {objCount}')
2190     return
2191
2192 // Process all entries of this subsection into xshFilebase entries.
2193 func f = B & i < objCount; len() -> bool:
2194     """f = B & i < objCount; len() method"""
2195     if err := parseReadFieldsEntry(s, xshFilebase, startOfB_Number); err == nil {
2196         return err
2197     }
2198     return nil
2199
2200 def log_Read_Println(self, subsectionHeader: str) -> None:
2201     """Log Read operation"""
2202     print(f'log_Read_Println({subsectionHeader}: end')
2203     return nil
2204
2205 // Create compressed object
2206 func compressObject(s string) (Object, error) -> Object:
2207     """compressObject method"""
2208     return nil
2209
2210 def log_Read_Println(self, compressedObject: Object) -> None:
2211     """Log Read operation"""
2212     print(f'log_Read_Println({compressedObject}: begin')
2213     return
2214
2215 def n_err = parseObject(s) -> bool:
2216     """n_err = parseObject(s) method"""
2217     if err == nil :
2218         return err
2219     else:
2220         return nil, err
2221
2222 def d_k, v = Dict() -> Dict:
2223     """d_k, v = Dict() method"""
2224     return nil
2225
2226 def r_return trivial Object: Integer, Array, etc. -> bool:
2227     """r_return trivial Object: Integer, Array, etc. method"""
2228     if ok {
2229         return true
2230     }
2231     log_Read_Println('compressedObject: end, any other than dict')
2232     return nil, nil
2233
2234 // Return trivial Object: Integer, Array, etc.
2235 func streamLength, streamLengthHeader = d.Length() -> bool:
2236     """streamLength, streamLengthHeader = d.Length() method"""
2237     if streamLength == nil || d.StreamLengthHeader == nil {
2238         return false
2239     }
2240     log_Read_Println('compressedObject: end, dict')
2241     return nil, nil
2242
2243 def return nil, errors.Msg(fmt.Sprintf("compressedObject: stream objects are not to be stored in an object stream")) -> bool:
2244     """return nil, errors.Msg(fmt.Sprintf("compressedObject: stream objects are not to be stored in an object stream")) method"""

```

```

340 // @formatter:off
341 } else {
342     cts.getTableObject(buider) = xdef.getTableEntry
343 }
344 }
345 }
346 }
347 }
348 }
349 }
350 }
351 }
352 }
353 }
354 }
355 }
356 }
357 }
358 }
359 }
360 }
361 }
362 }
363 }
364 }
365 }
366 }
367 }
368 }
369 }
370 }
371 }
372 }
373 }
374 }
375 }
376 }
377 }
378 }
379 }
380 }
381 }
382 }
383 }
384 }
385 }
386 }
387 }
388 }
389 }
390 }
391 }
392 }
393 }
394 }
395 }
396 }
397 }
398 }
399 }
400 }
401 }
402 }
403 }
404 }
405 }
406 }
407 }
408 }
409 }
410 }
411 }
412 }
413 }
414 }
415 }
416 }
417 }
418 }
419 }
420 }
421 }
422 }
423 }
424 }
425 }
426 }
427 }
428 }
429 }
430 }
431 }
432 }
433 }
434 }
435 }
436 }
437 }
438 }
439 }
440 }
441 }
442 }
443 }
444 }
445 }
446 }
447 }
448 }
449 }
450 }
451 }
452 }
453 }
454 }
455 }
456 }
457 }
458 }
459 }
460 }
461 }
462 }
463 }
464 }
465 }
466 }
467 }
468 }
469 }
470 }
471 }
472 }
473 }
474 }
475 }
476 }
477 }
478 }
479 }
480 }
481 }
482 }
483 }
484 }
485 }
486 }
487 }
488 }
489 }
490 }
491 }
492 }
493 }
494 }
495 }
496 }
497 }
498 }
499 }
500 }
501 }
502 }
503 }
504 }
505 }
506 }
507 }
508 }
509 }
510 }
511 }
512 }
513 }
514 }
515 }
516 }
517 }
518 }
519 }
520 }
521 }
522 }
523 }
524 }
525 }
526 }
527 }
528 }
529 }
530 }
531 }
532 }
533 }
534 }
535 }
536 }
537 }
538 }
539 }
540 }
541 }
542 }
543 }
544 }
545 }
546 }
547 }
548 }
549 }
550 }
551 }
552 }
553 }
554 }
555 }
556 }
557 }
558 }
559 }
560 }
561 }
562 }
563 }
564 }
565 }
566 }
567 }
568 }
569 }
570 }
571 }
572 }
573 }
574 }
575 }
576 }
577 }
578 }
579 }
580 }
581 }
582 }
583 }
584 }
585 }
586 }
587 }
588 }
589 }
590 }
591 }
592 }
593 }
594 }
595 }
596 }
597 }
598 }
599 }
600 }
601 }
602 }
603 }
604 }
605 }
606 }
607 }
608 }
609 }
610 }
611 }
612 }
613 }
614 }
615 }
616 }
617 }
618 }
619 }
620 }
621 }
622 }
623 }
624 }
625 }
626 }
627 }
628 }
629 }
630 }
631 }
632 }
633 }
634 }
635 }
636 }
637 }
638 }
639 }
640 }
641 }
642 }
643 }
644 }
645 }
646 }
647 }
648 }
649 }
650 }
651 }
652 }
653 }
654 }
655 }
656 }
657 }
658 }
659 }
660 }
661 }
662 }
663 }
664 }
665 }
666 }
667 }
668 }
669 }
670 }
671 }
672 }
673 }
674 }
675 }
676 }
677 }
678 }
679 }
680 }
681 }
682 }
683 }
684 }
685 }
686 }
687 }
688 }
689 }
690 }
691 }
692 }
693 }
694 }
695 }
696 }
697 }
698 }
699 }
700 }
701 }
702 }
703 }
704 }
705 }
706 }
707 }
708 }
709 }
710 }
711 }
712 }
713 }
714 }
715 }
716 }
717 }
718 }
719 }
720 }
721 }
722 }
723 }
724 }
725 }
726 }
727 }
728 }
729 }
730 }
731 }
732 }
733 }
734 }
735 }
736 }
737 }
738 }
739 }
740 }
741 }
742 }
743 }
744 }
745 }
746 }
747 }
748 }
749 }
750 }
751 }
752 }
753 }
754 }
755 }
756 }
757 }
758 }
759 }
760 }
761 }
762 }
763 }
764 }
765 }
766 }
767 }
768 }
769 }
770 }
771 }
772 }
773 }
774 }
775 }
776 }
777 }
778 }
779 }
780 }
781 }
782 }
783 }
784 }
785 }
786 }
787 }
788 }
789 }
790 }
791 }
792 }
793 }
794 }
795 }
796 }
797 }
798 }
799 }
800 }
801 }
802 }
803 }
804 }
805 }
806 }
807 }
808 }
809 }
810 }
811 }
812 }
813 }
814 }
815 }
816 }
817 }
818 }
819 }
820 }
821 }
822 }
823 }
824 }
825 }
826 }
827 }
828 }
829 }
830 }
831 }
832 }
833 }
834 }
835 }
836 }
837 }
838 }
839 }
840 }
841 }
842 }
843 }
844 }
845 }
846 }
847 }
848 }
849 }
850 }
851 }
852 }
853 }
854 }
855 }
856 }
857 }
858 }
859 }
860 }
861 }
862 }
863 }
864 }
865 }
866 }
867 }
868 }
869 }
870 }
871 }
872 }
873 }
874 }
875 }
876 }
877 }
878 }
879 }
880 }
881 }
882 }
883 }
884 }
885 }
886 }
887 }
888 }
889 }
890 }
891 }
892 }
893 }
894 }
895 }
896 }
897 }
898 }
899 }
900 }
901 }
902 }
903 }
904 }
905 }
906 }
907 }
908 }
909 }
910 }
911 }
912 }
913 }
914 }
915 }
916 }
917 }
918 }
919 }
920 }
921 }
922 }
923 }
924 }
925 }
926 }
927 }
928 }
929 }
930 }
931 }
932 }
933 }
934 }
935 }
936 }
937 }
938 }
939 }
940 }
941 }
942 }
943 }
944 }
945 }
946 }
947 }
948 }
949 }
950 }
951 }
952 }
953 }
954 }
955 }
956 }
957 }
958 }
959 }
960 }
961 }
962 }
963 }
964 }
965 }
966 }
967 }
968 }
969 }
970 }
971 }
972 }
973 }
974 }
975 }
976 }
977 }
978 }
979 }
980 }
981 }
982 }
983 }
984 }
985 }
986 }
987 }
988 }
989 }
990 }
991 }
992 }
993 }
994 }
995 }
996 }
997 }
998 }
999 }
1000 }

```

```

620 // If objectReference == null
621 // No object reference stream.
622 // If ctx.readHeader() de objectReferenceVersion() > 0 via ctx.readHeader()
623 // return null, errors.StreamClosed() (parseFailure() not a conformant reader
624 found incompatible v3, v4File, versionString())
625
626 // log.Read,Println("parseFailure() end")
627 // return offset, nil, error
628 // return offset, nil, error
629
630 // This file is using cross reference streams.
631
632 // If ctx.readHeader()
633 // ctx.readHeader() == true
634 // ctx.readHeader() == true
635
636 // If s conformant readers process hidden objects containing
637 // in thisReader before continuing to process any previous HiddenSection.
638 // ThisReader is expected to have new entries for hidden entries.
639 // May appear in HiddenSection only.
640
641 // If ctx.readHeader()
642 // s := s.parseHiddenSectionOfStream(objectReference, ctx); err == nil {
643 //     return nil, err
644 // }
645
646 // log.Read,Println("parseFailure() end")
647
648 // return offset, nil, error
649
650 // This file is using cross reference streams.
651
652 func scanInMaxA *bufio.Scanner (string, error) {
653     if s := s; s != nil {
654         if s.Err() != nil {
655             return "", s.Err()
656         }
657         return "", errors.New("pdcip: scanInMaxA: returning nothing")
658     }
659     return s.Text(), nil
660 }
661
662 func scanInMaxA *bufio.Scanner (ctx string, err error) {
663     for i := 0; i < 10; i++ {
664         s, err := scanInMaxA(s)
665         if err == nil {
666             return "", err
667         }
668         if !ctx(s) > 0 {
669             break
670         }
671     }
672
673     // Remove comment.
674     i := strings.Index(s, "\n")
675     if i > 0 {
676         s1 := s[:i]
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

[illegible]

```

1159 log.Debug.Println("readtable:Table: begin")
1160
1161 // Read table
1162 offset, err = offsetLastWeFSection(cts)
1163 if err == nil {
1164     return
1165 }
1166
1167 err = buildReadTableStartingAt(cts, offset)
1168 if err != nil {
1169     return errors.Wrap(err, "readtable:Table: unexpected error")
1170 }
1171
1172 if err == nil {
1173     return
1174 }
1175
1176 // log list of free objects (not the "free list").
1177 // log.Debug.Println("freeList: %v", cts.freeObjects)
1178
1179 // Iterate until freeList of objects.
1180 for cts.EnsureValidFreeList() {
1181     if err == nil {
1182         return
1183     }
1184 }
1185
1186 log.Debug.Println("readtable:Table: end")
1187
1188
1189
1190
1191 func growBuffer(buf []byte, size int, rd io.Reader) ([]byte, error) {
1192
1193     b := make([]byte, size)
1194
1195     // err = rd.Read(b)
1196     if err == nil {
1197         return nil, err
1198     }
1199
1200     //log.Debug.Println("growBuffer: Read b: %bytesin", n)
1201
1202     return append(buf, b...), nil
1203 }
1204
1205
1206 func nextStreamOffset(fileName string, streamLocalOff int) (off int) {
1207     off = streamLocal + len(fileName)
1208
1209     // Skip optional 4 bytes:
1210     // TODO: Should be skip optional whitespace instead?
1211     for ; lineOffset == 0x00; off++ {
1212     }
1213
1214     // Skip 8B hex.
1215     if lineOffset == "\n" {
1216         return
1217     }
1218
1219     // Size 8B hex.
1220 }

```

```

302         return index + 1, data[index], nil
303     }
304     // We have a full range, return terminated line.
305     return index + 1, data[0:index], nil
306 }
307 // case len(s) > 0:
308 // We have a full carriage return terminated line.
309 return index + 1, data[0:index], nil
310 }
311 // case len(s) == 0:
312 // We have a full newline-terminated line.
313 return index + 1, data[0:index], nil
314 }
315 }
316 // If we're at EOF, we have a final, non-terminated line. Return it.
317 if atEOF {
318     return len(data), data, nil
319 }
320 // Request more data.
321 return 0, nil, nil
322 }
323 // NonPositionedReader is a ReaderSeeker, offset +int64 (*bufio.Reader, error) {
324 func newNonPositionedReader(rs io.ReaderSeeker, offset +int64) (*bufio.Reader, error) {
325     if _, err := rs.Seek(offset, io.SeekStart); err != nil {
326         return nil, err
327     }
328     log.Printf("NonpositionedReader: positioned to offset: %d\n", offset)
329     return bufio.NewReader(rs), nil
330 }
331 // Get the file offset of the last WriteSection.
332 // Go to end of file and search backwards for the first occurrence of StartStr
333 // (offset may be negative)
334 func offsetLastWriteSection(ctx *Context) (+int64, error) {
335     rs := ctx.Readers
336     var {
337         prevBuf, words []byte
338         bufSize int64 = 512
339         offset int64
340     }
341     for i := 1; offset == 0; i++ {
342         if _, err := rs.Seek(-int64(i)*bufSize, io.SeekEnd);
343             err == nil {
344                 return nil, errors.New("p4proc: can't find last seek section")
345             }
346         log.Printf("Scanning for offsetLastWriteSection starting at %d\n", off)
347         curBuf := make([]byte, bufSize)
348     }

```

[illegible]

```

547 // return nil, err
548 }
549
550 // Parse the stream and return "stream" before "endobj" if "endobj" within
551 // "stream"
552 // There is no guarantee that "endobj" is contained in this buffer for large
553 // streams
554 // If streamEnd < 0 (endobj > 0) endobj < streamEnd()
555 // return nil, errors.New(fmt.Sprintf("parseStream: corrupt pdf file"))
556
557 // Init object parse buf
558 // Line = lineStream
559 objectNumber, generationNumber, err := parseObjectAttributes(l)
560 if err == nil {
561     return nil, err
562 }
563
564 // parse this object
565 log.Read.Print("parseStream: xrefst objid %d gen%d\n", objectNumber,
566 generationNumber)
567 log.Read.Print("parseStream: dereferencing object %d\n", objectNumber)
568 if err := parseObject(l);
569 if err == nil {
570     return nil, errors.New(fmt.Sprintf("parseStream: no object"))
571 }
572
573 log.Read.Print("parseStream: we have an object: %d\n", o)
574
575 streamOffset := offset
576 if err := xRefStreamInit(o, o, objectNumber, streamOffset)
577 if err == nil {
578     return nil, err
579 }
580
581 // We have an xref stream object
582 err = parseTableInit(o, dict, ctx, xRefTable)
583 if err == nil {
584     return nil, err
585 }
586
587 // Parse xRefStream and create xRefTable entries for embedded objects.
588 err = extractTableEntries(xRefStreamStreamID.Content, sd, ctx)
589 if err == nil {
590     return nil, err
591 }
592
593 // Create xRefTable entries for xRefStreamDict.
594 entry :=
595     xRefTableEntry{
596         Name:      false,
597         Offset:    offset,
598         Generation: generationNumber,
599         Object:    sd
600     }

```

```

7870 return s1, nil
7871
7872 func Index(s string) (bool, error) {
7873     s_err = parseOptions()
7874     if err == nil {
7875         return false, err
7876     }
7877     _, ok := os.Stderr.(*os.File)
7878     return ok, nil
7879 }
7880
7881 func scantrailer(s *bufio.Scanner, line string) (string, error) {
7882     var buf bytes.Buffer
7883     var err error
7884     var i, j, k int
7885     log.Read.Print("(line: %s)\n", line)
7886
7887     // Scan for dict start tag "dict"
7888     for {
7889         i = strings.Index(line, "=")
7890         if i >= 0 {
7891             k = i
7892             line, err = scanline(s)
7893             log.Read.Print("(line: %s)\n", line)
7894             if err == nil {
7895                 return "", err
7896             }
7897         }
7898     }
7899
7900     line = line[i:]
7901     buf.WriteString(line)
7902     buf.WriteString("\n")
7903     log.Read.Print("scantrailer dictstart after start tag: %s\n", line)
7904
7905     // Scan for dict end tag "}" but account for inner dicts.
7906     line = line[i:]
7907     for {
7908         if len(line) == 0 {
7909             line, err = scanline(s)
7910             if err == nil {
7911                 return "", err
7912             }
7913             buf.WriteString(line)
7914             buf.WriteString("\n")
7915             log.Read.Print("scantrailer dictstart next line: %s\n", line)
7916         }
7917         i = strings.Index(line, "=")
7918         if i >= 0 {
7919             j = i
7920             line = strings.Index(line, "}")
7921             if j >= 0 {

```

```

3920         if $! == $null {
3921             echoCount = 1
3922         } else if ($? == $null) {
3923             echoCount = 1
3924         } if ($?) == $null {
3925             echoCount = 2
3926         }
3927     } else {
3928         return nil, $, errorCorruptHeader
3929     }
3930 }
3931
3932 log.Read.Print("header version: end, found header version: $d\$, pdfVersion")
3933
3934 return pdfVersion, echoCount, nil
3935 }
3936
3937 // sha256sum is a hack for digesting corrupt xref sections.
3938 // It populates the sha256 by reading in all indirect objects line by line
3939 // and then computes the sha256 of a single xref section - meaning no incremental
3940 // updates have been seen.
3941 func sha256sum(xrefInfo Content) error {
3942     var xref lists
3943     g := &ContentDecoder{
3944         ctx: &Context{
3945             Proc: true,
3946             Offset: 82,
3947             Generation: 10,
3948         },
3949     }
3950     rs := &Readers{
3951         echoCount, &ctx.Read, &Count
3952     }
3953     var off, offEnd int64
3954
3955     rd, err := newHashListBuilder(xrefInfo, &offset)
3956     if err == nil {
3957         return err
3958     }
3959     s := bufio.NewScanner(rd)
3960     s.Split(lexer)
3961
3962     for {
3963         tok := s.Text()
3964         if tok == "[Byte]" {
3965             var {
3966                 withIndex bool
3967                 withHeader bool
3968                 withTrailer bool
3969             }
3970
3971             for {
3972                 line, err := scanLineRaw(s)
3973                 if err == nil {
3974                     break
3975                 }
3976                 if withHeader {
3977                     offset += int64(len(line)) + echoCount
3978                 }
3979                 if withTrailer {
3980                     b := append(b, '\n')
3981                     b := append(b, line...)
3982                     b := append(b, '\n')
3983                 }
3984                 if s.String().IndexLine, "startzard" {
3985                     if ! z o {

```

[illegible][illegible]

```

13620200: // @param objArray = objArray
13620201:
13620202: log_Read_Printf(m "para%0bjectStream end" )
13620203:
13620204: return nil
13620205:
13620206:
13620207: // for each object embedded in this @ReadStream create the corresponding obj table
13620208:
13620209: 406: func extractTableEntriesFromReadStream(buf []byte, xid int64, result int, ctx
13620210: Context) error {
13620211:
13620212: log_Read_Printf("extractTableEntriesFromReadStream begin")
13620213:
13620214: // @note:
13620215: // A value of zero for any element in the M array indicates that the corresponding
13620216: // field shall not be present in the stream.
13620217: // And the default value shall be zero, if there is one.
13620218: // If the first element is zero, the type field shall not be present, and shall
13620219: // default to type 1.
13620220:
13620221: 415: i1 := xid.M[0]
13620222: i2 := xid.M[1]
13620223: i3 := xid.M[2]
13620224:
13620225: 419: xrefEntryLen := i1 + i2 + i3
13620226:
13620227: log_Read_Printf("extractTableEntriesFromReadStream: begin xrefEntryLen=%d",
13620228: xrefEntryLen)
13620229:
13620230: 422: if len(buf[:xrefEntryLen]) > 0 {
13620231: return errors.New("%p: corrupt tableEntriesFromReadStream: corrupt"
13620232: )
13620233: }
13620234:
13620235: 426: objCount := len(obj.Objects)
13620236:
13620237: log_Read_Printf("extractTableEntriesFromReadStream: objCount=%d xid=%d",
13620238: objCount, xid.Objects)
13620239:
13620240: 429: log_Read_Printf("extractTableEntriesFromReadStream: len(buf):%d
13620241: len(xrefEntryLen):%d", len(buf), objCount+xrefEntryLen)
13620242:
13620243: 438: if len(buf) < xrefEntryLen {
13620244: // @note: there is an insufficient xref entry not accounted for by "index".
13620245: // @note: shall ignore such a message and do not treat this as an error.
13620246: return errors.New("%p: extractTableEntriesFromReadStream: corrupt"
13620247: )
13620248: }
13620249:
13620250: 443: xrefEntryLen :=
13620251:
13620252: 445: j := 0
13620253:
13620254: // @note: interprets the content of buf as an index.
13620255: bufIndex := func(buf []byte) (i int) {
13620256:
13620257: 448:
13620258: 449: i := 0
13620259:
13620260: 452: i := 0
13620261:
13620262: 454: i := len(buf)
13620263:
13620264: 456:
13620265: 457: return
13620266:
13620267: 459:
13620268: 460:

```

```

617 log.Read.Println("params:Stream: Insert new shFileable entry for Object %s",
618 objChumber)
619
620 ctx.Table<objChumber> = Entry
621 ctx.Read.ParamsStreamId = objChumber + 1 true
622 prevOffset = s.d.PreviousOffset
623
624 log.Read.Println("params:Stream: end")
625
626 return prevOffset, nil
627
628 // func readParamsStreamId() (shFileable *File, error) {
629 func paramsStreamIdStream() offset int64, ctx *Context, error {
630 func paramsStreamIdStream() begin() {
631
632 rd, err = newPositionedReader(ctx.readHeader, offset)
633 if err == nil {
634     return err
635 }
636
637 // err = paramsStreamIdStream, offset, ctx)
638 if err == nil {
639     return err
640 }
641
642 log.Read.Println("params:Stream: end")
643
644 return nil
645 }
646
647 // func readParamsStreamId() (shFileable *File, error) {
648 func paramsTrailerInfo() (Dict, shFileable *File) error {
649
650 log.Read.Println("params:TrailerInfo begin")
651
652 if v, found = d.Find("encrypt"); found {
653     encryptObjId = d.IndirectEntry("encrypt")
654     if encryptObjId == nil
655         shFileable.Params = encryptObjId
656         log.Read.Println("params:TrailerInfo: Encrypt object: %s",
657 shFileable.Params)
658     }
659 }
660
661 if shFileable.Size == nil {
662     size = d.Size()
663     if size == nil {
664         return errors.New("params:TrailerInfo: missing entry 'Size'")
665     }
666     // Not reusable
667     // params after all read in.
668     shFileable.size = size
669 }
670
671 if shFileable.Hash == nil {
672     hashObjId = d.IndirectEntry("hash")
673 }

```

```

645 // 100
646         if k == 0 {
647             // Check for dice
648             ok, err := isDice(buf.String())
649             if err == nil || ok {
650                 return buf.String(), nil
651             }
652         } else {
653             k--
654         }
655         line = line[j+2:]
656         continue
657     }
658     // done
659     line, err = scanLine(s)
660     if err == nil {
661         return "", err
662     }
663     buf.WriteString(line)
664     buf.WriteString(" ")
665     log.Read_Fmt("scan trailer dicthub next line: %s\n", line)
666 } else {
667     // 100
668     s = string.Index(line, "\n")
669     if s < 0 {
670         // done
671         k++
672         line = line[j+2:]
673     } else {
674         // done
675         if i + j {
676             // handle oc
677             k++
678             line = line[j+2:]
679         } else {
680             // handle so
681             // Check for dice
682             ok, err = isDice(buf.String())
683             if err == nil || ok {
684                 return buf.String(), nil
685             }
686         } else {
687             //
688             //
689             line = line[j+2:]
690         }
691     }
692 }
693 }
694 }
695 }
696 }
697 }
698
699 func processTrailer(ctx Context, s bufio.Scanner, line string) (uint64, error) {
700     // line trailer string
701     if line == "trailer" {
702         trailerStr := line[7:]
703         log.Read_Fmt("process trailer trailer leftover: %s\n", trailerStr)
704     }
705 }

```

[illegible]

```

3132 //
3133 line = string(buf, "nomod");
3134 endof = strings.Index(line, "nomod");
3135 streamend = strings.Index(line, "stream");
3136 //
3137 if endof > 0 && (streamend < 0 && streamend > endof) {
3138     // no stream marker in buf detected.
3139     break
3140 }
3141 //
3142 // For very rare cases where "stream" also occurs within obj dict
3143 // (e.g. "stream" as a key), we have to check for "stream" as a value.
3144 for streamid := 0 < keystrings.IndexAfter(endof, line, line) {
3145     lastStreamAddr := strings.Index(line, "stream", streamid)
3146 }
3147 //
3148 log.Read.Printf("buffer: endof=%d stream=%d\n", endof, streamid)
3149 //
3150 if streamend > 0 {
3151     // streamOffset ... the offset where the actual stream data begins
3152     // streamOffset is right after the eof after "stream"
3153     streamOffset = endof + 1
3154 } else {
3155     // slack is 10 // for optional whitespace + eof (max 2 bytes)
3156     need = streamend + len("stream") + slack
3157     if len(line) < need {
3158         // to prevent buffer overflow.
3159         buf, err = growBuf(buf, need-len(line), 10)
3160         if err == nil {
3161             return nil, 0, 0, 0, err
3162         }
3163     }
3164     line = string(buf)
3165     streamOffset = last(strings.IndexAfter(line, streamend))
3166 }
3167 //
3168 // Log Read.Panic("buffer: end, returned bufio.Reader.ReadString(), len(buf),
3169 // streamOffset, nil
3170 return buf, endof, streamend, streamOffset, nil
3171 }
3172 //
3173 // Return true if "stream" follows end of dict; otherwise, false
3174 func isStreamEnd(streamid int, buf []byte, streamend int) bool {
3175     //
3176     log.Read.Panic("isStreamEnd(streamid=%d, buf, streamend) not called", streamid)
3177     //
3178     // Get a slice of the chunk right in front of "stream".
3179     b := buf[streamend:]
3180     //
3181     // Look for last end of dict marker.
3182     end := strings.LastIndex(b, ">")
3183     if end < 0 {
3184         // No end of dict in buf.
3185     }

```

